PCA

Jyotishka Datta\ Department of Statistics, Virginia Tech

February 17, 2026

Source

- A lot of R code in this demo is adapted from the excellent book *An Introduction to Statistical Learning with Applications in R* by James, Witten, Hastie, and Tibshirani. The book website provides R code, labs, and other resources.
- The Crime and Iris data examples were influenced by materials from Dr. Sumanta Basu. PCR code & examples were provided by Dr. Bobby Gramacy.

Recap (PCA)

Let $\mathbf{X}$ be an $n \times p$ data matrix with columns centered (column means subtracted).

First principal component (PC1) is the direction $\boldsymbol{\beta}_1 = (\beta_{11}, \ldots, \beta_{p1})^\top$ that maximizes the variance of the projected data, subject to unit length:

$$\max_{\boldsymbol{\beta}_1} \frac{1}{n} \sum_{i=1}^{n} (\mathbf{x}_i^\top \boldsymbol{\beta}_1)^2 \quad \text{s.t.} \quad \|\boldsymbol{\beta}_1\|_2 = 1.$$

The scores are $y_{i1} = \mathbf{x}_i^\top \boldsymbol{\beta}_1$.

Second PC

The second principal component β_2 solves

$$\max_{\beta_2} \frac{1}{n}\sum_{i=1}^{n}(\mathbf{x}_i^\top \beta_2)^2 \quad \text{s.t.} \quad \|\beta_2\|_2 = 1,\ \beta_1^\top \beta_2 = 0.$$

Scores y_{i1} and y_{i2} are uncorrelated by construction.

Optimization for PCA

Two equivalent computational routes for PCA:

- **SVD of $\mathbf{X}$**: $\mathbf{X} = \mathbf{U}\,\mathbf{\Sigma}\,\mathbf{V}^\top$. The columns of $\mathbf{V}$ are PC loading vectors β_j; singular values give standard deviations of PCs.
- **Eigen-decomposition of covariance**: On centered data, $\mathbf{X}^\top\mathbf{X}$ eigenvectors are loadings, eigenvalues are variances of PCs.

Using R

Two common functions:

- `prcomp()` uses an SVD on the (centered/scaled) data matrix (numerically stable).
- `princomp()` uses an eigen-decomposition of the covariance (or correlation) matrix.

US Arrests data

Description. Statistics in arrests per 100,000 residents for Assault, Murder, and Rape in each of the 50 US states in 1973. Also given is percent of the population in urban areas (`UrbanPop`).

```
states <- row.names(USArrests)
states
```

```
##  [1] "Alabama"        "Alaska"         "Arizona"        "Arkansas
##  [5] "California"     "Colorado"       "Connecticut"    "Delaware
##  [9] "Florida"        "Georgia"        "Hawaii"         "Idaho"
## [13] "Illinois"       "Indiana"        "Iowa"           "Kansas"
## [17] "Kentucky"       "Louisiana"      "Maine"          "Marylan
## [21] "Massachusetts"  "Michigan"       "Minnesota"      "Mississi
## [25] "Missouri"       "Montana"        "Nebraska"       "Nevada"
## [29] "New Hampshire"  "New Jersey"     "New Mexico"     "New York
## [33] "North Carolina" "North Dakota"   "Ohio"           "Oklahoma
## [37] "Oregon"         "Pennsylvania"   "Rhode Island"   "South Ca
## [41] "South Dakota"   "Tennessee"      "Texas"          "Utah"
## [45] "Vermont"        "Virginia"       "Washington"     "West Vir
## [49] "Wisconsin"      "Wyoming"
```

PCA on USArrests

```r
pr.out <- prcomp(USArrests, scale. = TRUE)
names(pr.out)
```

```
## [1] "sdev"     "rotation" "center"   "scale"    "x"
```

```r
pr.out$center
```

```
##   Murder  Assault UrbanPop     Rape
##    7.788  170.760   65.540   21.232
```

```r
pr.out$scale
```

```
##    Murder   Assault  UrbanPop      Rape
##  4.355510 83.337661 14.474763  9.366385
```

Loadings and scores

```r
pr.out$rotation  # loadings (each column is a PC loading vector)
```

```
##                  PC1        PC2        PC3         PC4
## Murder   -0.5358995 -0.4181809  0.3412327  0.64922780
## Assault  -0.5831836 -0.1879856  0.2681484 -0.74340748
## UrbanPop -0.2781909  0.8728062  0.3780158  0.13387773
## Rape     -0.5434321  0.1673186 -0.8177779  0.08902432
```

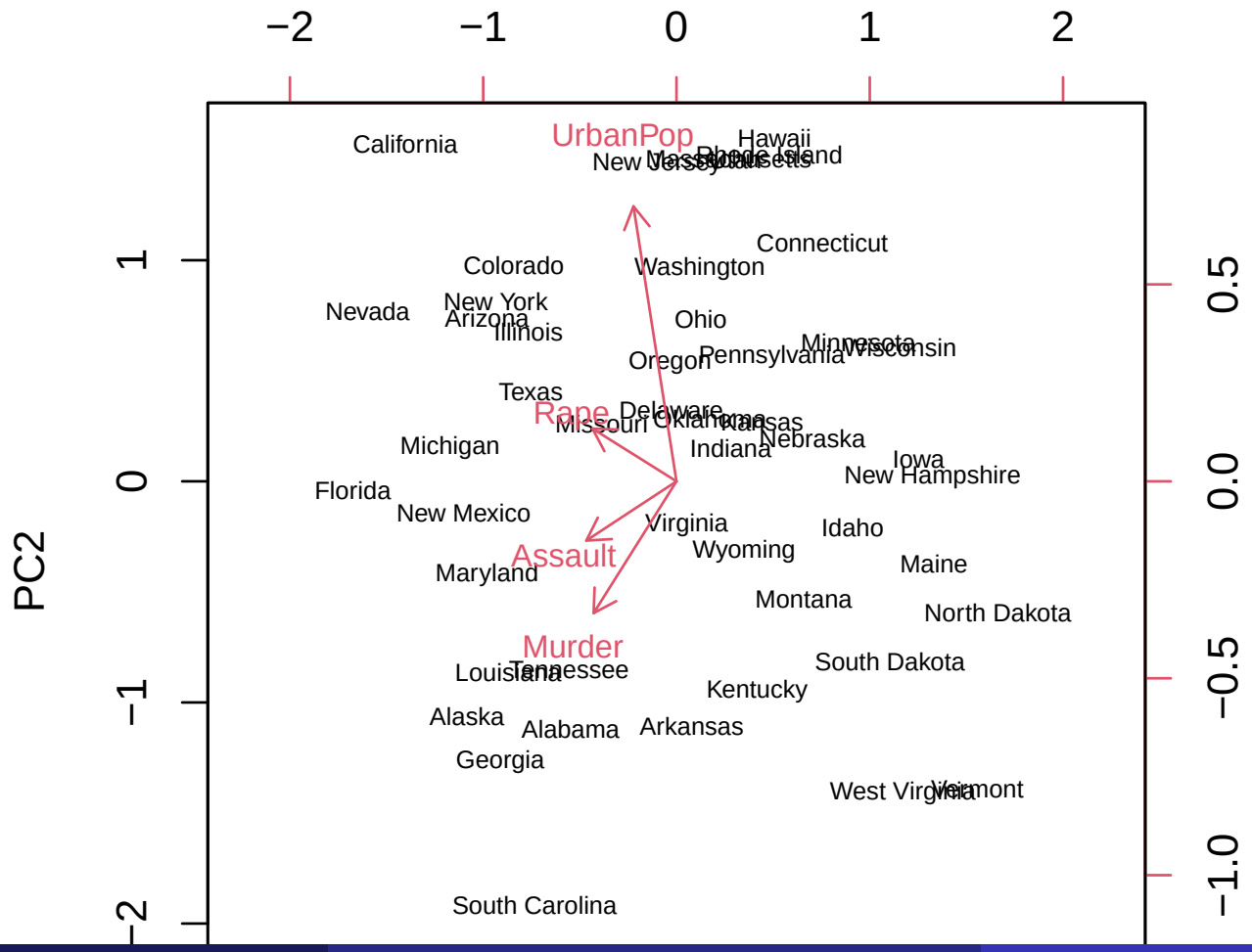
```r
dim(pr.out$x)    # 50 observations x 4 PCs
```

```
## [1] 50  4
```

Biplot

A biplot visualizes both observations (scores) and variables (loadings).

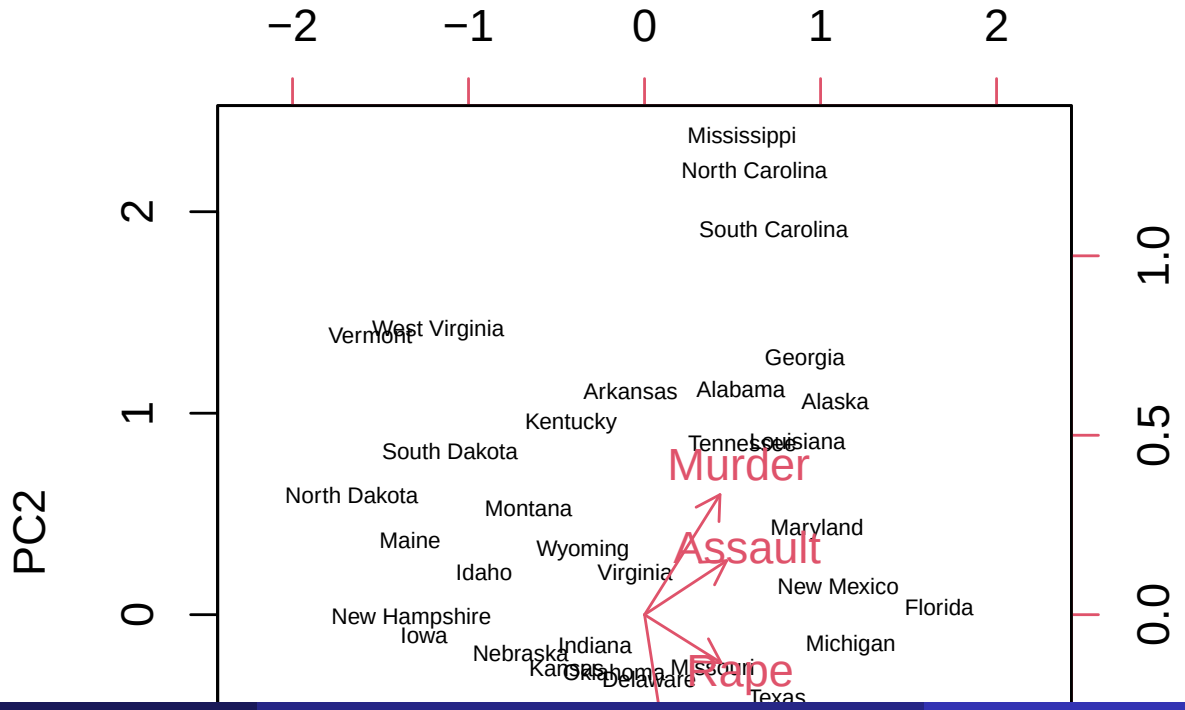
```
biplot(pr.out, scale = 0, cex = c(0.6, 0.75), xlim = c(-4, 4))
```



Sign indeterminacy

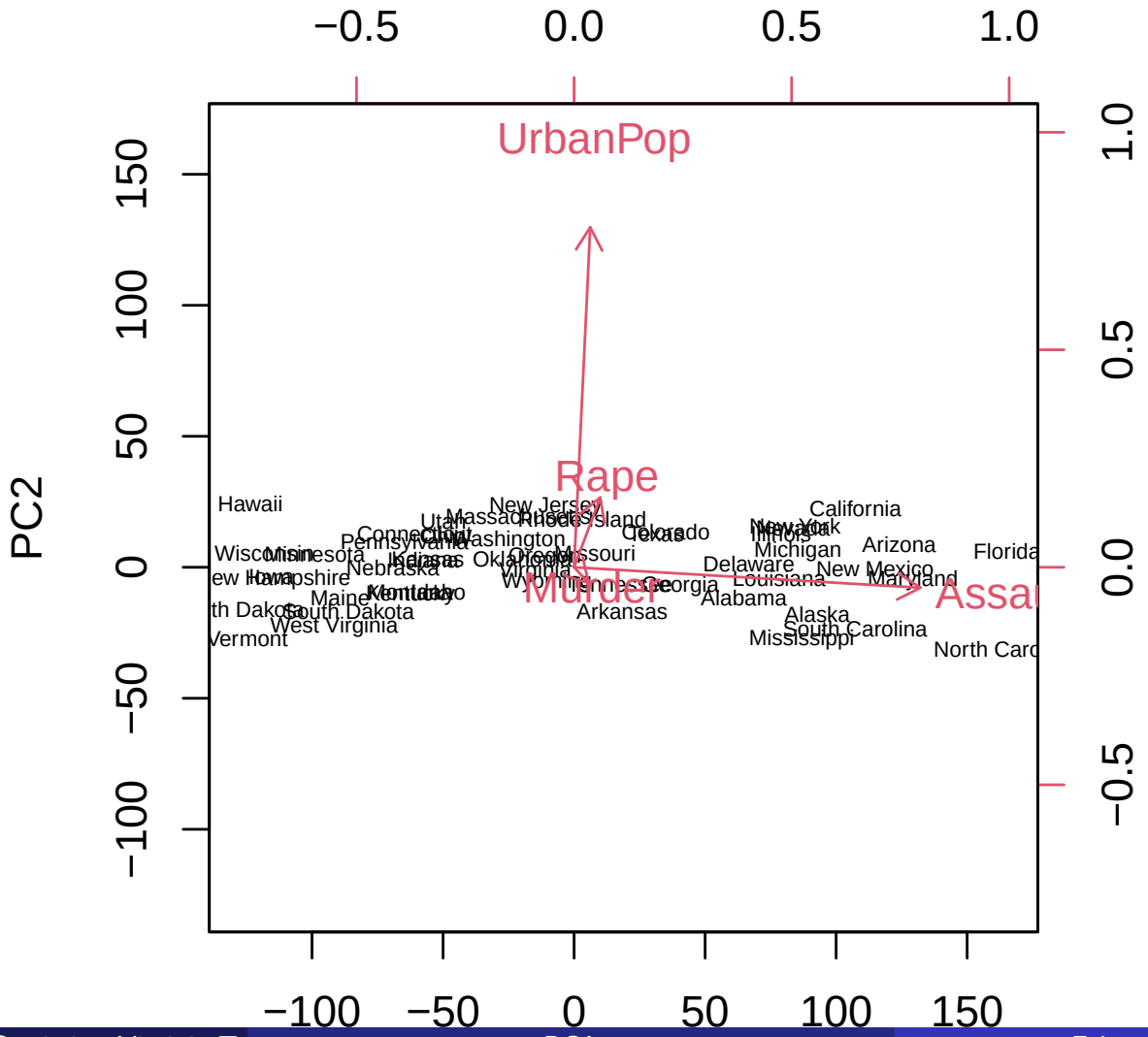
Principal components are unique **up to sign**. Flipping the sign of a loading vector and its scores leaves the geometry unchanged.

```
pr.tmp <- pr.out
pr.tmp$rotation <- -pr.tmp$rotation
pr.tmp$x <- -pr.tmp$x
biplot(pr.tmp, scale = 0, cex = c(0.5, 1), xlim = c(-4, 4))
```



Importance of scaling

```
biplot(prcomp(USArrests, scale. = FALSE), scale = 0, cex = c(0.5, 1)
```



Standard deviations and PVE

```r
pr.out$sdev
```

```
## [1] 1.5748783 0.9948694 0.5971291 0.4164494
```

```r
pr.var <- pr.out$sdev^2
pr.var
```

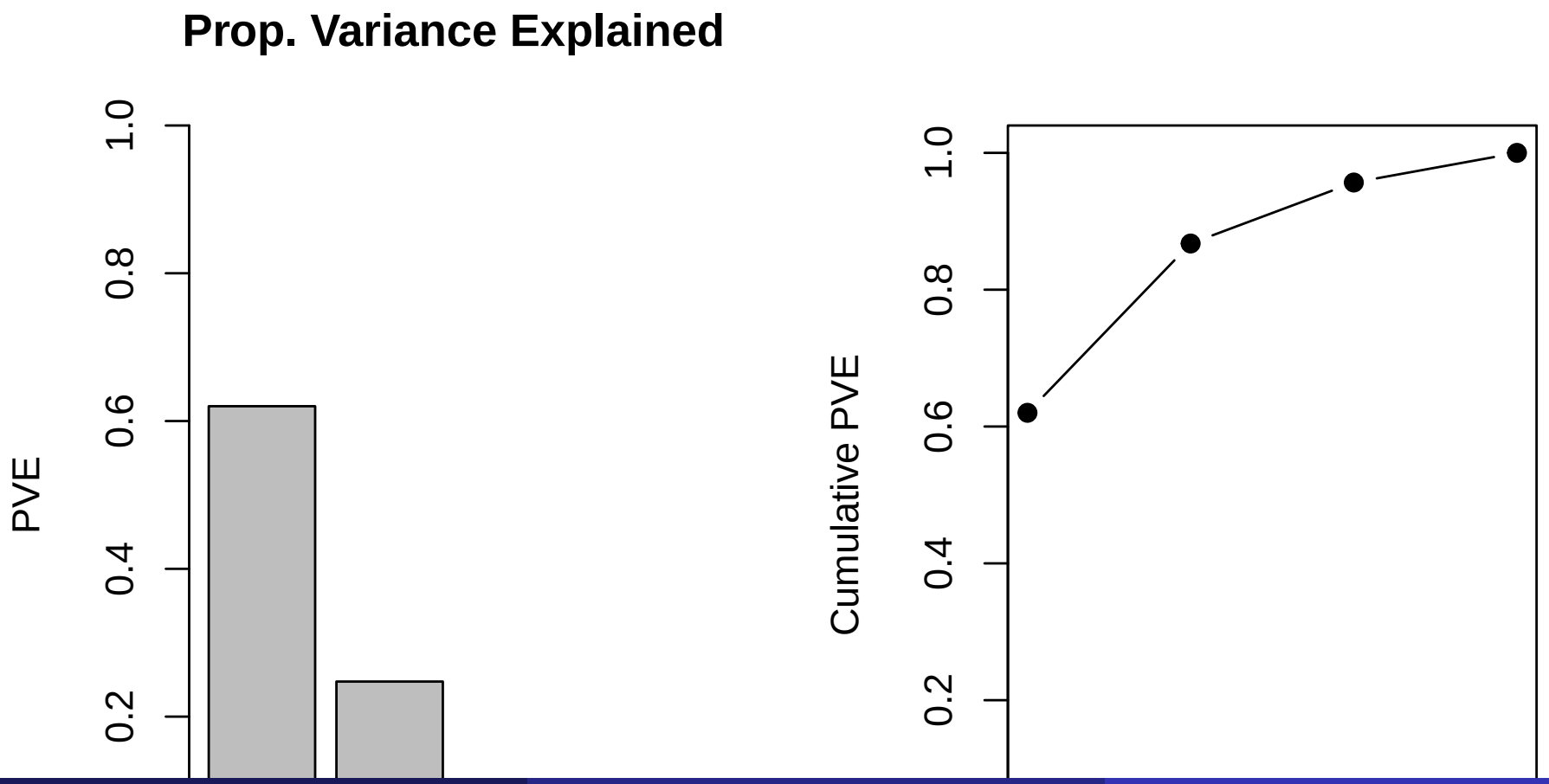
```
## [1] 2.4802416 0.9897652 0.3565632 0.1734301
```

```r
pve <- pr.var / sum(pr.var)
pve
```

```
## [1] 0.62006039 0.24744129 0.08914080 0.04335752
```

Proportion of Variance Explained (PVE)

```
par(mfrow = c(1, 2))
barplot(pve, main = "Prop. Variance Explained", ylab = "PVE", ylim =
plot(cumsum(pve), type = "b", pch = 19, xlab = "Number of PCs", ylab
     ylim = c(0, 1))
```



More examples (princomp)

prcomp (SVD on data) and princomp (eigen on covariance/correlation) usually agree closely; some prefer prcomp for numerical stability.

PCA for City Crime data

The city crime dataset contains crime rates per 100,000 people for 72 large US cities in 1994. Variables: Murder, Rape, Robbery, Assault, Burglary, Larceny, Motor Vehicle Theft.

```
# Place a file named "city_crime.txt" in the project directory.
# The file should have 7 numeric columns in the order listed above.
crime <- NULL
if (file.exists("city_crime.txt")) {
  crime <- tryCatch({
    read.table("city_crime.txt", header = TRUE)
  }, error = function(e) tryCatch({
    read.table("city_crime.txt", header = FALSE)
  }, error = function(e2) NULL))
}
if (is.null(crime)) {
  message("city_crime.txt not found or unreadable; subsequent crime
}
```

Scatterplot matrix (crime)

```r
pairs(crime)
```

PCA on crime data (princomp)

```r
pca.crime <- princomp(scale(crime), cor = FALSE)
summary(pca.crime)
loadings(pca.crime)
```

PVE and scree (crime)

```r
PoV <- pca.crime$sdev^2 / sum(pca.crime$sdev^2)
barplot(PoV, main = "Prop Var Exp")
screeplot(pca.crime, type = "line")
```

Scores and biplot (crime)

```
pcs.crime <- predict(pca.crime)
plot(pcs.crime[, 1:2], type = "n", xlab = '1st PC', ylab = '2nd PC')
text(pcs.crime[, 1:2], row.names(crime))
biplot(pca.crime, cex = c(0.5, 1))
```

Questions

1. Which cities appear most safe? Which are most crime-prone?
2. How might we interpret high/low values on the second PC?

Subtle difference: scaling in prcomp vs princomp

```
pc.cr  <- prcomp(crime, scale = TRUE, center = TRUE)
pc.cr1 <- princomp(crime, cor = TRUE, scores = TRUE)
pc.cr$scale
pc.cr1$scale
```

Iris data

We will check if PCA can still detect species after adding noise variables.

```r
data(iris)
str(iris)
```

```
## 'data.frame':    150 obs. of  5 variables:
##  $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
##  $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
##  $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
##  $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
##  $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1
```

```r
# Matrix of original 4 features
X_iris <- data.matrix(iris[, 1:4])
# Add 6 columns of i.i.d. N(0,1) noise as extra dimensions
set.seed(2023)
noise <- matrix(rnorm(nrow(X_iris) * 6), ncol = 6)
# Combine so that noise appears as first 6 columns
dat.iris <- cbind(noise, X_iris)
colnames(dat.iris) <- c(paste0('V', 1:6), colnames(X_iris))
# Standardize before PCA on correlation structure
```

Iris PCA: summary and scree

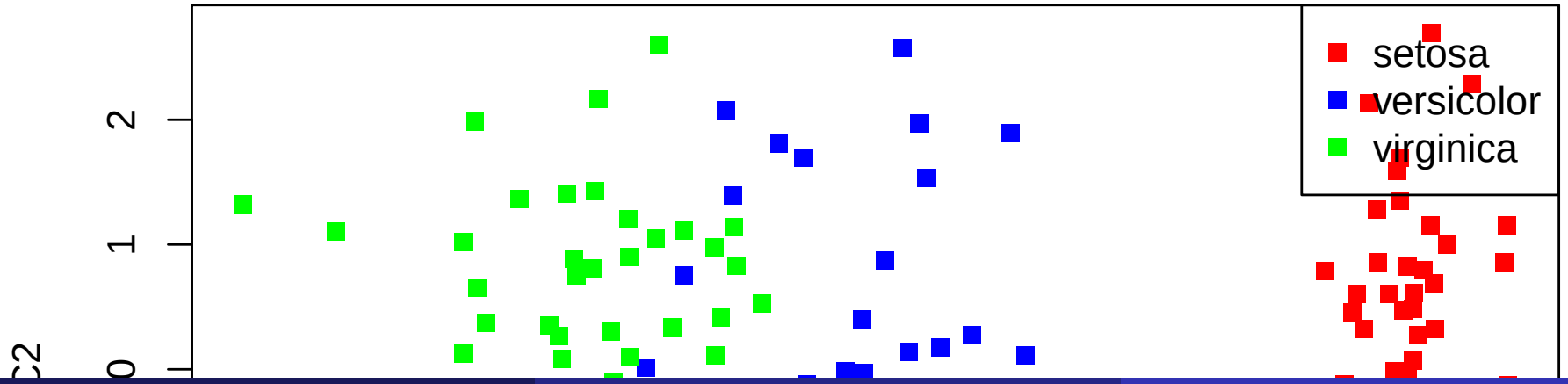
```r
summary(pca.iris)
```

```
## Importance of components:
##                          Comp.1    Comp.2    Comp.3     Comp.4
## Standard deviation     1.7255297 1.1567711 1.1219589 0.98784253
## Proportion of Variance 0.2977453 0.1338119 0.1258792 0.09758329
## Cumulative Proportion  0.2977453 0.4315572 0.5574364 0.65501966
##                            Comp.6     Comp.7     Comp.8     Comp.
## Standard deviation     0.93381666 0.88361291 0.83775736 0.3612829
## Proportion of Variance 0.08720136 0.07807718 0.07018374 0.0130525
## Cumulative Proportion  0.83666113 0.91473831 0.98492205 0.9979745
```

```r
screeplot(pca.iris, type = "line")
```

Iris PC scores and colors

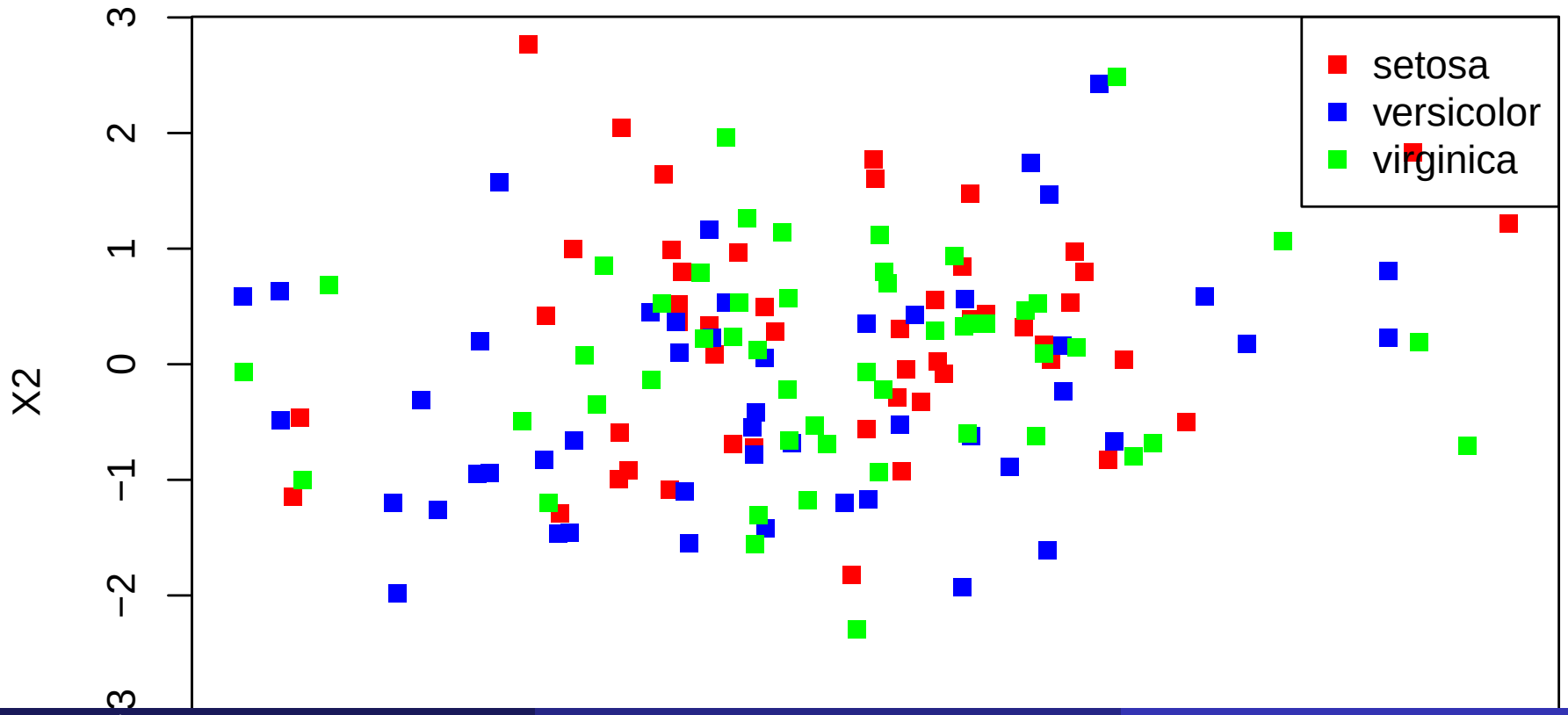
```
pcs.iris <- predict(pca.iris)
col.code <- as.character(iris$Species)
col.code[col.code == "setosa"]     <- 'red'
col.code[col.code == "versicolor"] <- 'blue'
col.code[col.code == "virginica"]  <- 'green'
plot(pcs.iris[, 1:2], type = 'p', pch = 15, col = col.code,
     xlab = "PC1", ylab = "PC2")
legend("topright", col = c('red','blue','green'),
       legend = c('setosa','versicolor','virginica'), pch = 15)
```



Why PCs can help

Compare to plotting just two (arbitrary) variables in the original space.

```
plot(dat.iris[, 1:2], type='p', pch=15, col=col.code, xlab="X1", yla
legend("topright", col=c('red','blue','green'), legend=c('setosa','v
```



Principal Components Regression (PCR)

PCR uses the leading PC score columns $z_1, \ldots, z_m$ (with $m \ll p$) as predictors in a regression for response y. With orthogonal PCs, the OLS fit decomposes into a sum of univariate regressions in the PC scores.

PCR: backing out coefficients

Each PC is a linear combination of the original (scaled) variables, so PCR coefficients in the original feature space can be recovered from the PC loadings and the regression on PC scores.

Toy PCR example (olive oil)

```r
# Ensure pls is available
if (!requireNamespace("pls", quietly = TRUE)) {
  install.packages("pls", repos = "https://cloud.r-project.org")
}
library(pls)
data(oliveoil)
# Response: first sensory score; Predictors: chemical measurements
set.seed(1)
y <- as.numeric(oliveoil$sensory[, 1])
X <- as.matrix(oliveoil$chemical)
# Standardize predictors
for (j in 1:ncol(X)) X[, j] <- X[, j] / sd(X[, j])
# PCA via eigen decomposition of covariance
S  <- cov(X)
eS <- eigen(S, symmetric = TRUE)
Z  <- X %*% eS$vectors
colnames(Z) <- paste0("PC", 1:ncol(Z))
# Fraction of variance explained
cumsum(eS$values) / sum(eS$values)
```

PCR via pls::pcr() and CV

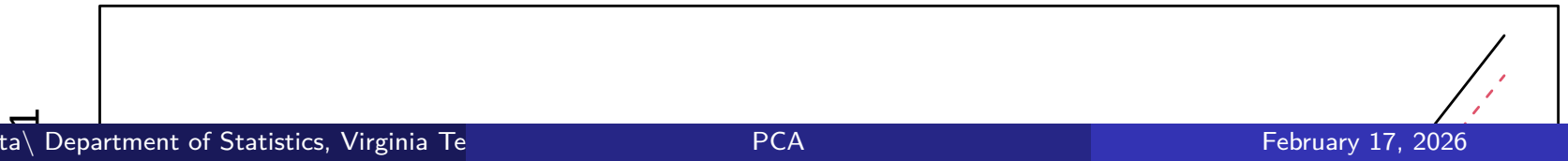
```
fit.pcr <- pcr(y ~ X, ncomp = 3)
rbind(automated = coef(fit.pcr, intercept = TRUE), byhand = as.numer
```

```
##                [,1]      [,2]     [,3]      [,4]      [,5]     [,6]
## automated 129.103 -5.406939 -1.34047 -2.905699 -8.539748 1.091108
## byhand    129.103 -5.406939 -1.34047 -2.905699 -8.539748 1.091108
```

```
cv.pcr <- pcr(y ~ X, validation = "LOO")
RMSEP(cv.pcr)
```

```
##        (Intercept)  1 comps  2 comps  3 comps  4 comps  5 comps
## CV            20.1    16.31    16.11    17.73     19.1    21.73
## adjCV         20.1    16.26    16.10    17.56     18.9    21.42
```

```
plot(RMSEP(cv.pcr), main = "")
```



Reproducibility notes

- Rendering this file to PDF requires a LaTeX installation. If you use R, you can install TinyTeX via:

```
if (!requireNamespace("tinytex", quietly = TRUE)) install.packag
```

- To include the city crime example, add `city_crime.txt` to the working directory.
- Tested with R ž265 4.0 and packages in CRAN.